Report: Optimising NYC Taxi Operations

Include your visualisations, analysis, results, insights, and outcomes. Explain your methodology and approach to the tasks. Add your conclusions to the sections.

1. Data Preparation

1.1. Loading the dataset

We are loading the data using pandas Library.

```
df=pd.read_parquet('../Datasets and Dictionary/trip_records/2023-1.parquet')
df.info()
df.head()
```

1.1.1. Sample the data and combine the files

Sampling the data by taking 0.78% of each hour's data for each day, so that the total number of records is limited to 300,000.

Looping through all 12 Parquet files and creating the final DataFrame using the above sampling technique.

```
# Loop through dates and then loop through every hour of each date
for date in sortedDates:
    dailyData= filteredDf[filteredDf["tpep_pickup_datetime"].dt.date==date]
    hours=dailyData["tpep_pickup_datetime"].dt.hour.unique()
    sortedHours=sorted(hours)

    # Iterate through each hour of the selected date
    for hour in sortedHours:
        hourly_data=dailyData[dailyData["tpep_pickup_datetime"].dt.hour==hour]

        # Sample 5% of the hourly data randomly
        sample=hourly_data.sample(frac=0.0078,random_state=42)# Only sampling 0.78%

        # add data of this hour to the dataframe
        sampled_data=pd.concat([sampled_data,sample])

# Concatenate the sampled data of all the dates to a single dataframe
df = pd.concat([df,sampled_data])# we initialised this empty DF earlier
```

Creating the new parquet file from the final data frame.

```
# Store the df in csv/parquet
# df.to_parquet('')
df.to_parquet('Sampled2023TripRecords.parquet')
```

2. Data Cleaning

Loading the data from file which was created in first step.

```
# Load the new data file
df=pd.read_parquet('Sampled2023TripRecords.parquet')
```

2.1. Fixing Columns

2.1.1. Fix the index

Fixing the index by using reset_index function.

```
df["month"]=df["tpep_pickup_datetime"].dt.month
df=df.sort_values("month").reset_index(drop=True)
df.drop(["month","store_and_fwd_flag"],axis=1,inplace=True)
```

2.1.2. Combine the two airport_fee columns

Combining both airport_fee columns using the combine_first method, which selects the non-NaN values.

```
df['combined_airport_fee']=  df['airport_fee'].combine_first(df['Airport_fee'])
```

Drop the “airport_fee” and “Airport_fee” column from the dataframe.

```
df.drop(['airport_fee','Airport_fee'],axis=1,inplace=True)
```

Renaming the ‘combined_airport_fee’ to ‘airport_fee’

```
df=df.rename(columns={'combined_airport_fee':'airport_fee'})
```

2.2. Handling Missing Values

2.2.1. Find the proportion of missing values in each column

Calculating the mean of null values for all the columns

`df.isnull().mean(axis=0)*100`

Here is the list of missing values corresponding to each column

```
VendorID                 0.00000
tpep_pickup_datetime     0.00000
tpep_dropoff_datetime    0.00000
passenger_count          3.35286
trip_distance            0.00000
RatecodeID               3.35286
PULocationID             0.00000
DOLocationID             0.00000
payment_type             0.00000
fare_amount              0.00000
extra                    0.00000
mta_tax                  0.00000
tip_amount               0.00000
tolls_amount             0.00000
improvement_surcharge    0.00000
total_amount             0.00000
congestion_surcharge     3.35286
airport_fee              3.35286
dtype: float64
```

2.2.2. Handling missing values in passenger_count

Calculating the mode of the passenger_count variable and replacing the null values with it.

```
# Display the rows with null values
print(df[df['passenger_count'].isnull()].head())
# Impute NaN values in 'passenger_count'
df['passenger_count'].value_counts(normalize=True) # getting the value_counts for passenger_count
df['passenger_count'].mode() # checking the mode of passenger_count
df.loc[df['passenger_count'].isnull(),'passenger_count']=1
```

Removed the rows where passenger_count and fare_amount is 0

```
#Removing the Above Data points as passenger_count is 0 and fare_amount
df= df[~((df['passenger_count']==0) & (df['fare_amount']==0))]
```

Imputing the mode value wherever passenger_count is 0

```
# Imputing the mode value for passenger count wherever it is 0
df.loc[df['passenger_count']==0,['passenger_count']]=1
```

2.2.3. Handle missing values in RatecodeID

Imputing the mode value for RatecodeID wherever it is null, as it is a categorical column.

```
mode=df['RatecodeID'].mode()[0]
```

```
df.loc[df['RatecodeID'].isnull(),['RatecodeID']]=mode
```

2.2.4. Impute NaN in congestion_surcharge

Using the median to impute the congestion_surcharge, as it is preferred over the mean.

```
# handle null values in congestion_surcharge
median=df['congestion_surcharge'].median() # Selecting the median for imputing conge

df.loc[df['congestion_surcharge'].isnull(),['congestion_surcharge']]=median
```

Imputing the median value for “airport_fee” column as well

```
# Handle any remaining missing values
airportFeeMedian=df['airport_fee'].median()
df.loc[df['airport_fee'].isnull(),['airport_fee']]=airportFeeMedian
```

2.3. Handling Outliers and Standardising Values

2.3.1. Check outliers in payment type, trip distance and tip amount columns

Removing the enteries where passenger_count>6

```
# remove passenger_count > 6
df=df[~(df['passenger_count']>6)]
```

Removing enteries where trip_distance is near to 0 and fare amount is greater than equal to 300

```
df=df[~((df["trip_distance"]==0) & (df['fare_amount']>=300))]
```

Removing the rows where trip_distance and fare_amount is 0 but the drop and pickup location are different

```
# Removing the rows where trip_distance and fare_amount is 0 but the drop and pickup location are different
df=df[~((df["trip_distance"]==0) & (df['fare_amount']==0)  & (df['PULocationID']!=df['DOLocationID']))]
```

Removing the rows where trip distance is more than 250 miles

```
df=df[~(df["trip_distance"]>250)]
```

Imputing the payment_type with the mode where ever it is 0

```
paymentTypeMode=df['payment_type'].mode()[0]
df.loc[df['payment_type']==0,['payment_type']]=paymentTypeMode
```

Creating a standaridised column for tip_amount using the below code

```
meanTipAmt=df['tip_amount'].mean()
```

```
stdTipAmt=df['tip_amount'].std()
```

```
df["standardised_tip_amount"]=(df['tip_amount']-meanTipAmt)/stdTipAmt
```

3. Exploratory Data Analysis

3.1. General EDA: Finding Patterns and Trends

3.1.1. Classify variables into categorical and numerical

These are Categorical and Numerical Variable

Categorical variable

- VendorID
- tpep_pickup_datetime
- tpep_dropoff_datetime
- RatecodeID
- PULocationID
- DOLocationID
- payment_type
- pickup_hour

Numerical variable (Facts/Measure)

- trip_duration
- trip_distance
- passenger_count
- fare_amount
- extra
- mta_tax
- tip_amount
- tolls_amount
- improvement_surcharge
- total_amount
- congestion_surcharge
- airport_fee

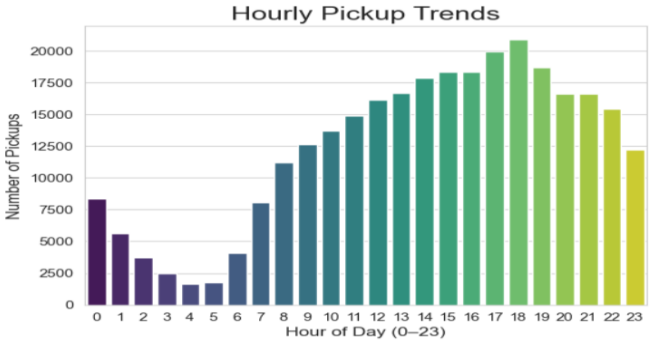
3.1.2. Analyse the distribution of taxi pickups by hours, days of the week, and months

Taxi Pickups by hours

Code:

```
# Find and show the hourly trends in taxi pickups
df['pickup_hour']=df["tpep_pickup_datetime"].dt.hour
hourly_counts=df['pickup_hour'].value_counts().sort_index()
sns.set_style("whitegrid")
sns.barplot(x=hourly_counts.index, y=hourly_counts.values, palette="viridis")
plt.title('Hourly Pickup Trends', fontsize=18)
plt.xlabel('Hour of Day (0-23)', fontsize=12)
plt.ylabel('Number of Pickups', fontsize=12)
plt.xticks(range(0, 24))
plt.show()
```

Graph:



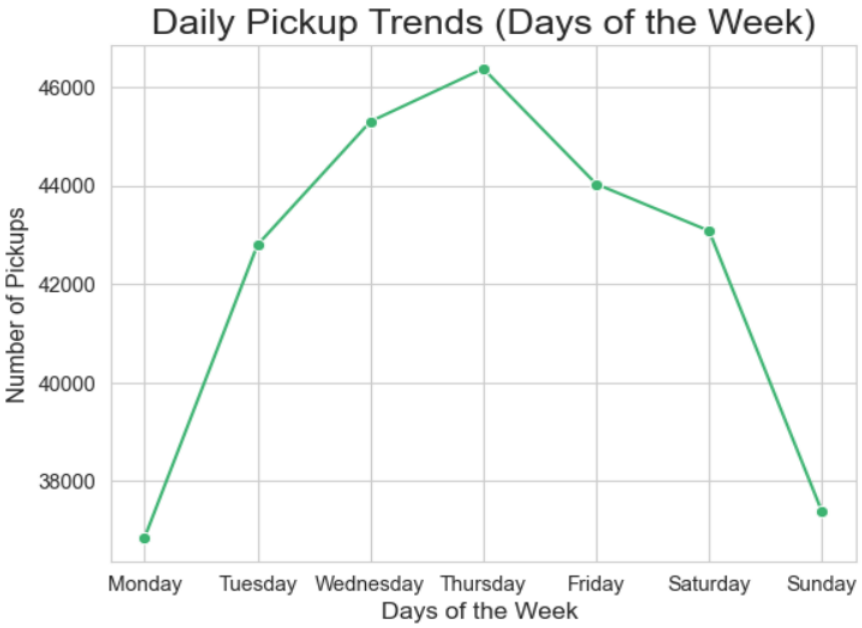
Outcome: Shows the taxi demand is more in day hours (9:00AM to 6PM)

Taxi pickups by days of the week

Code:

```
df['pickup_day_of_the_week']=df['tpep_pickup_datetime'].dt.day_of_week
dayCounts=df['pickup_day_of_the_week'].value_counts().sort_index()
sns.lineplot(x=dayCounts.index, y=dayCounts.values, marker='o', color='mediumseagreen')

# Add title and labels
plt.title('Daily Pickup Trends (Days of the Week)', fontsize=18)
plt.xlabel('Days of the Week ', fontsize=12)
plt.ylabel('Number of Pickups', fontsize=12)
plt.xticks(ticks=dayCounts.index, labels=['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday'])
plt.show()
```

Graph: (Line Graph)



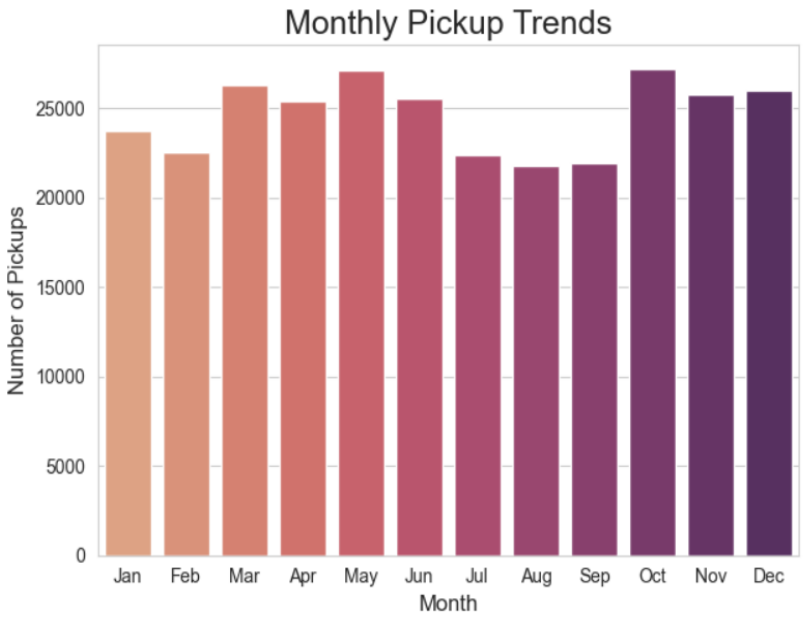
Outcome: The taxi demand is high on Tuesday Wednesday and Friday as compared to other days

Taxi Pickups by Month

Code:

```
month_labels = ['Jan','Feb','Mar','Apr','May','Jun','Jul','Aug','Sep','Oct','Nov','Dec']
df['monthly_pickups']=df['tpep_pickup_datetime'].dt.month
monthlyCounts=df['monthly_pickups'].value_counts().sort_index()
sns.barplot(x=monthlyCounts.index, y=monthlyCounts.values, palette="flare")
plt.xticks(ticks=range(0, 12), labels=month_labels)
plt.title('Monthly Pickup Trends', fontsize=18)
plt.xlabel('Month', fontsize=12)
plt.ylabel('Number of Pickups', fontsize=12)
plt.tight_layout()
plt.show()
```

Graph: (Bar Plot)



Outcome: Shows the demand is same across the year

3.1.3. Filter out the zero/negative values in fares, distance and tips

Checking the count of zero in the above column

```
zeroValues=df[['fare_amount','tip_amount','total_amount','trip_distance']]==0
zeroValues.sum(axis=0)
```

These are the counts of zero's in the above column

```
fare_amount         81
tip_amount       67951
total_amount        34
trip_distance     5825
dtype: int64
```

Finally filtering out the zero values from the above column

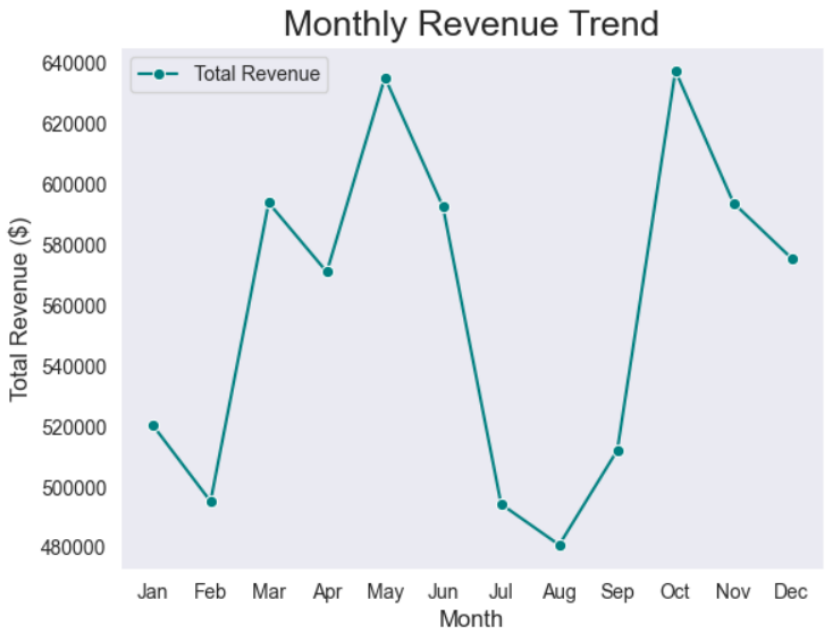
```
# Create a df with non zero entries for the selected parameters.
new_df=df[~((df['fare_amount']==0)|(df['tip_amount']==0)|(df['total_amount']==0)|(df['trip_distance']==0))]
```

3.1.4. Analyse the monthly revenue trends

Code:

```
monthlyRevenue=new_df.groupby('monthly_pickups')['total_amount'].sum().sort_index()
sns.set_style("dark")
sns.lineplot(x=monthlyRevenue.index, y=monthlyRevenue.values, marker='o', color='teal', label='Total Revenue')
plt.xticks(ticks=range(1, 13), labels=month_labels)
plt.title('Monthly Revenue Trend', fontsize=18)
plt.xlabel('Month', fontsize=12)
plt.ylabel('Total Revenue ($)', fontsize=12)
plt.show()
```

Graph: (Line Plot)



Outcome: Shows the maximum revenue was in Month of May and October

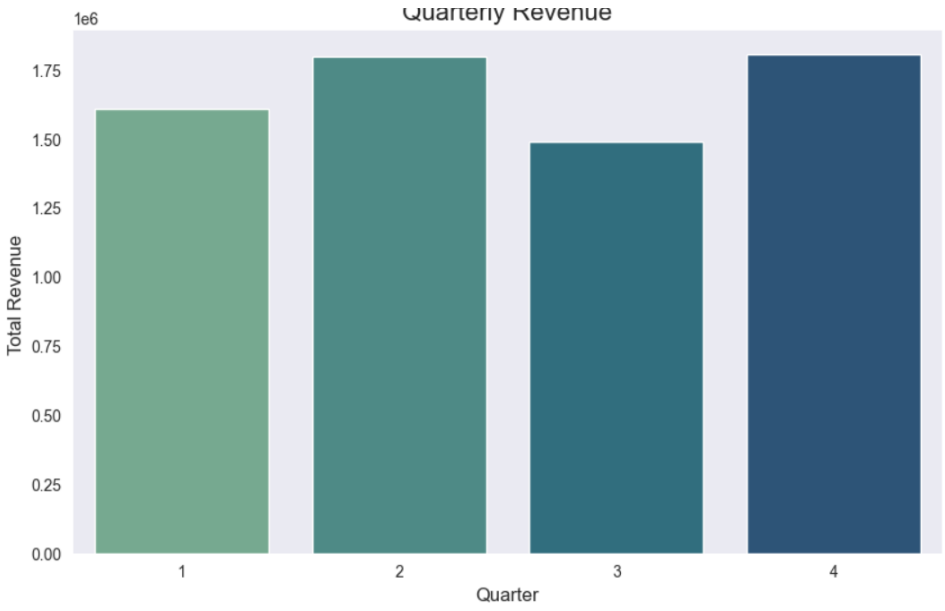
3.1.5. Find the proportion of each quarter's revenue in the yearly revenue

First calculating the proportion of revenue and modelling into Graph for better understanding

Code:

```
new_df['quarter_pickups']=df['tpep_pickup_datetime'].dt.quarter
quarterRevenue=new_df.groupby('quarter_pickups')['total_amount'].sum().sort_index()
quarterRevenue.values
sns.set_style("dark")
plt.figure(figsize=(10, 6))
sns.barplot(x=quarterRevenue.index, y=quarterRevenue.values, palette="crest")
plt.title('Quarterly Revenue', fontsize=16)
plt.xlabel('Quarter', fontsize=12)
plt.ylabel('Total Revenue ', fontsize=12)
plt.show()
```

Graph:



Outcome: Displays the revenue per quarter, with peak values observed in Q2 and Q4.

3.1.6. Analyse and visualise the relationship between distance and fare amount

Code:

```
# Show how trip fare is affected by distance
sns.scatterplot(data=new_df, x='trip_distance', y='fare_amount', alpha=0.5, color='purple')
# Add title and labels
plt.title('Relationship Between Trip Distance and Fare Amount', fontsize=18)
plt.xlabel('Trip Distance (miles)', fontsize=12)
plt.ylabel('Fare Amount ', fontsize=12)
plt.show()
```

Graph:

Analysis: Shows that trip distance and fare amount are highly correlated, meaning that as the distance increases, the fare also increases.

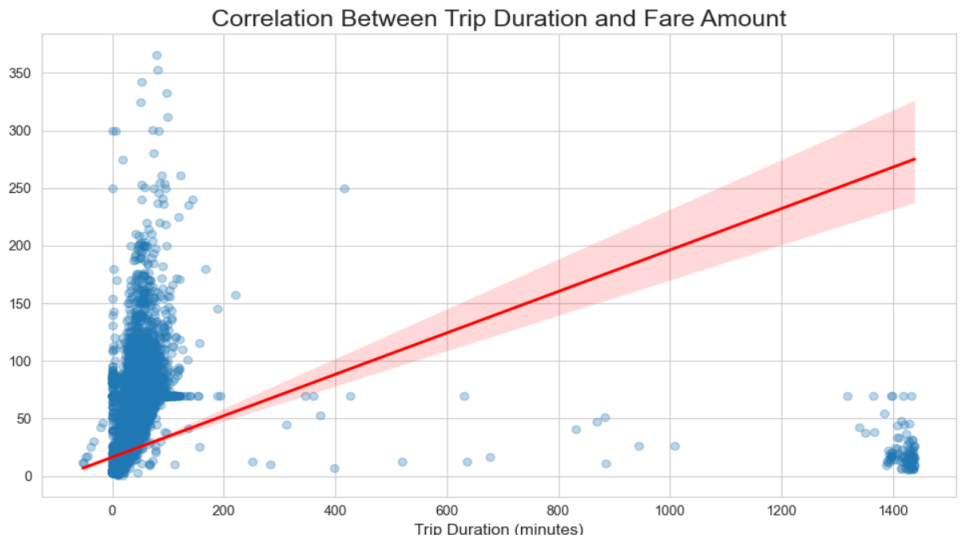
3.1.7. Analyse the relationship between fare/tips and trips/passengers

Fare and trip duration

Code:

```
new_df['trip_duration'] = (new_df['tpep_dropoff_datetime'] - new_df['tpep_pickup_datetime']).dt.total_seconds() / 60 #in minutes
sns.set_style("whitegrid")
plt.figure(figsize=(10, 6))
sns.regplot(data=new_df, x='trip_duration', y='fare_amount',scatter_kws={'alpha': 0.3}, line_kws={'color': 'red'})
plt.title('Correlation Between Trip Duration and Fare Amount', fontsize=18)
plt.xlabel('Trip Duration (minutes)', fontsize=12)
plt.ylabel('Fare Amount ', fontsize=12)
plt.tight_layout()
plt.show()
```

Graph:



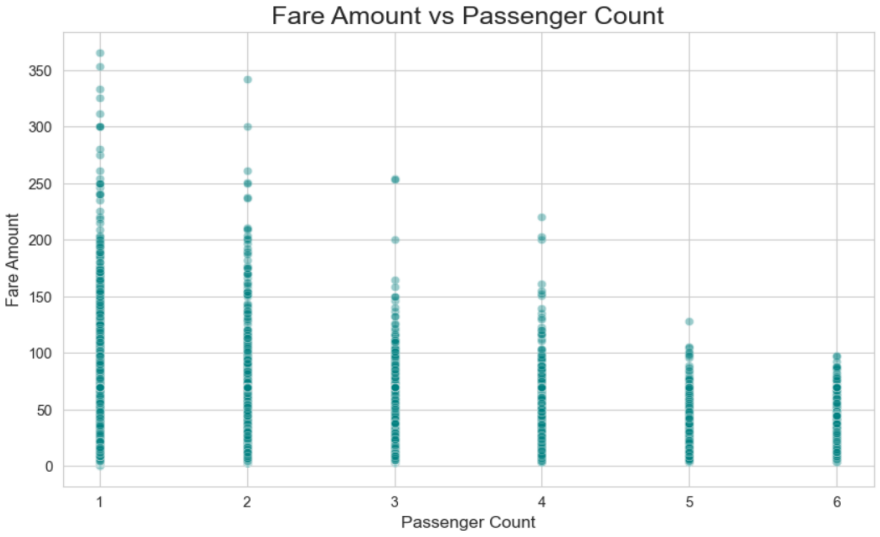
Results: Correlation is 0.34

Fare and passenger_count

Code:

```
sns.set_style("whitegrid")
plt.figure(figsize=(10, 6))
sns.scatterplot(data=new_df, x='passenger_count', y='fare_amount', alpha=0.4, color='teal')
plt.title('Fare Amount vs Passenger Count', fontsize=18)
plt.xlabel('Passenger Count', fontsize=12)
plt.ylabel('Fare Amount', fontsize=12)
plt.show()
```

Graph:



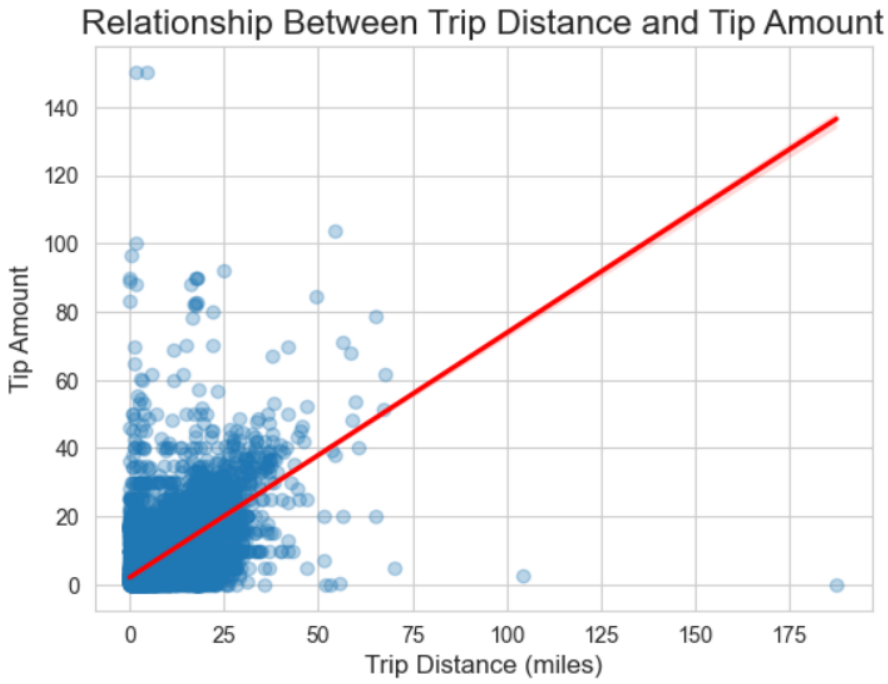
Result: Correlation is 0.03

Tip amount and trip distance

Code:

```
# Show relationship between tip and trip distance
sns.regplot(data=new_df, x='trip_distance', y='tip_amount',scatter_kws={'alpha': 0.3}, line_kws={'color': 'red'})
plt.title('Relationship Between Trip Distance and Tip Amount', fontsize=16)
plt.xlabel('Trip Distance (miles)', fontsize=12)
plt.ylabel('Tip Amount', fontsize=12)
plt.show()
```

Graph:



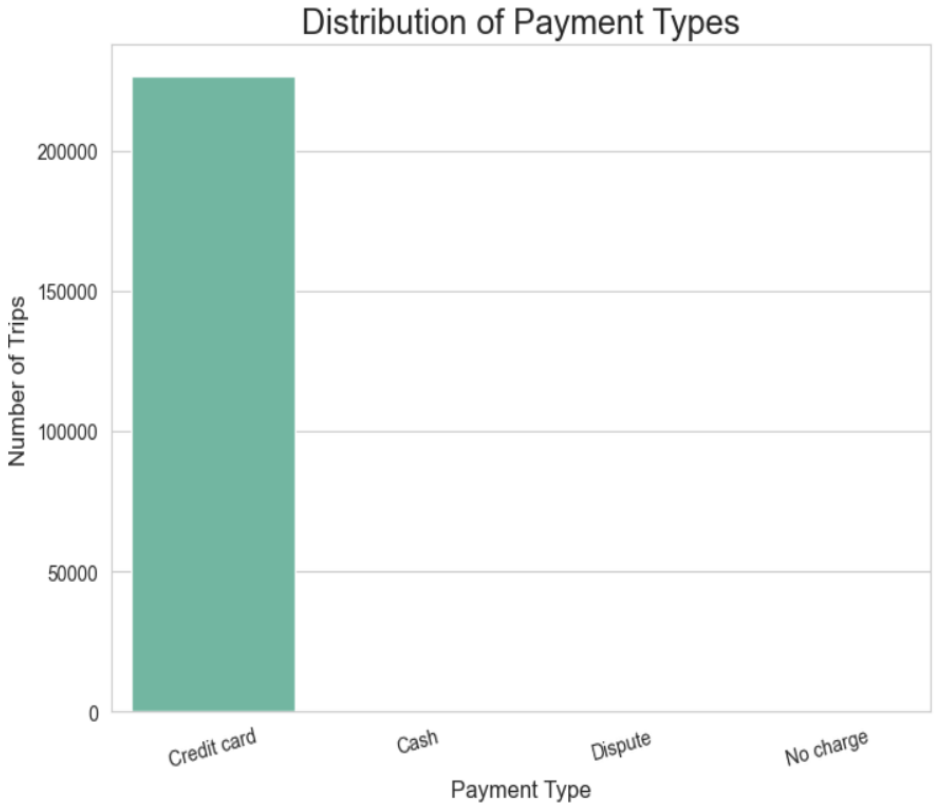
Result: Correlation is 0.79

3.1.8. Analyse the distribution of different payment types

Code:

```
paymentMap={1: 'Credit card',
2: 'Cash',
3: 'No charge',
4: 'Dispute',
5: 'Unknown',
6: 'Voided trip'}
new_df['payment_type_label']=new_df['payment_type'].map(paymentMap)
sns.set_style("whitegrid")
plt.figure(figsize=(8, 6))
sns.countplot(data=new_df, x='payment_type_label', palette='Set2')
plt.title('Distribution of Payment Types', fontsize=18)
plt.xlabel('Payment Type', fontsize=12)
plt.ylabel('Number of Trips', fontsize=12)
plt.xticks(rotation=15)
plt.show()
```

Graph:



Outcome: Shows Credit card as the preferred payment mode

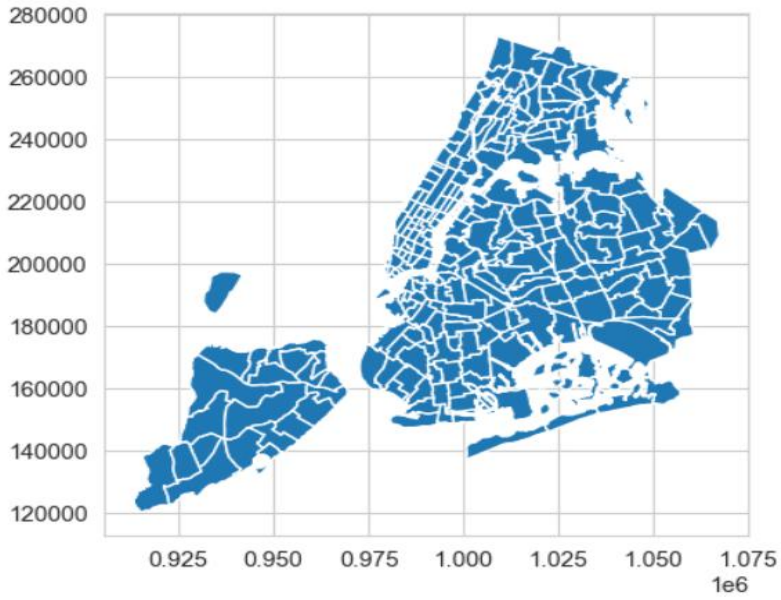
3.1.9. Load the taxi zones shapefile and display it

First installed the geo pandas for visualizing the shape files.

Loading the Geo file using

```
import geopandas as gpd

# Read the shapefile using geopandas

shapefile_path ="../taxi_zones/taxi_zones.shp"

zones = gpd.read_file(shapefile_path)#  read the .shp file using gpd

zones.head()
```

Displaying the data as map using zones.plot()



3.1.10. Merge the zone data with trips data

Pandas merge function is used to merge both the zone data and trips data.

Created the new data frame “merge_df” which has the joined data for both data Frames

```
merge_df=pd.merge(new_df, zones, left_on='PULocationID', right_on='LocationID', how='inner')
merge_df.head()
```

3.1.11. Find the number of trips for each zone/location ID

Used value_counts for getting the trips per location ID

```
# Group data by location and calculate the number of trips
tripsByLocation=merge_df['LocationID'].value_counts().sort_index()
print("Trips by Location \n",tripsByLocation)
```

```
Trips by Location
 LocationID
1         4
4       255
6         1
7        52
9         1
        ...
258       1
260      19
261    1128
262    3160
263    4477
Name: count, Length: 177, dtype: int64
```

3.1.12. Add the number of trips for each zone to the zones dataframe

Created “tripsByLocation” pandas series for adding trips to the zones data frame

```
zones['trips']=zones['LocationID'].map(tripsByLocation)
```

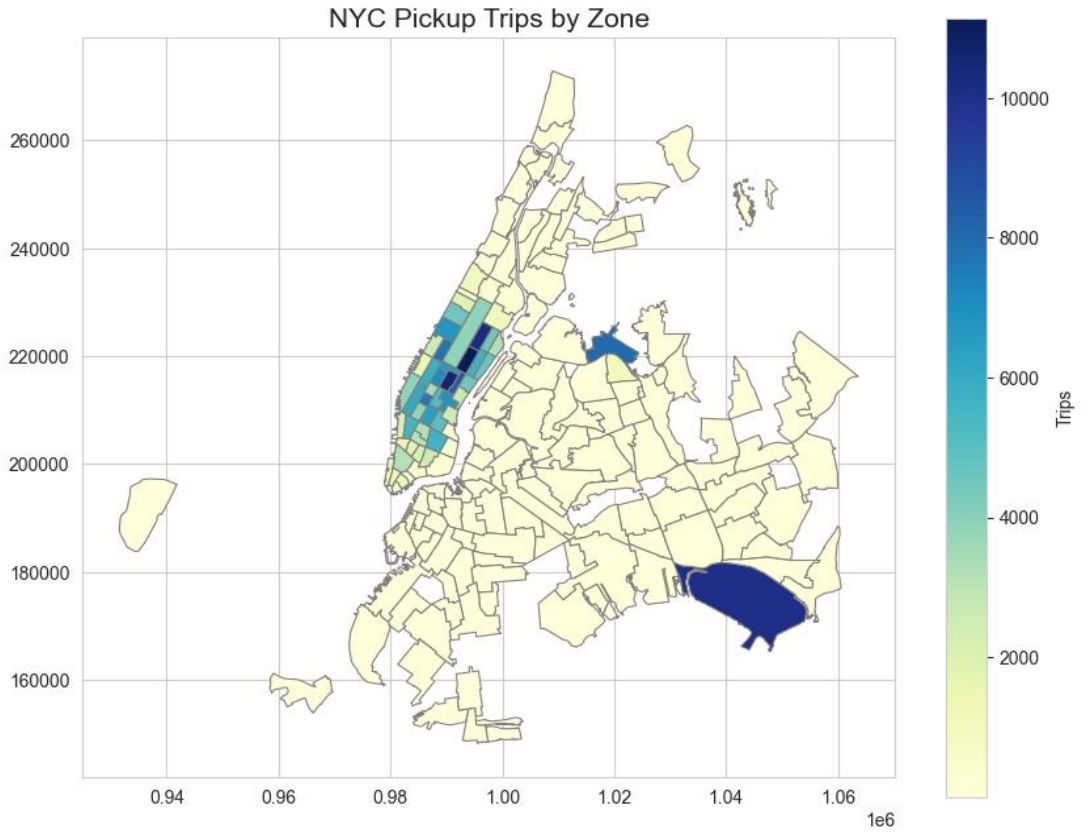
3.1.13. Plot a map of the zones showing number of trips

First, we define the figure and axes for the map then we use plot function to plot the Map.

Code:

```
# Define figure and axis
fig, ax=plt.subplots(1,1,figsize=(10,8))

# Plot the map and display it
zones.plot(column='trips',cmap='YlGnBu',linewidth=0.8,edgecolor='grey',legend=True,ax=ax,legend_kwds={'label':"Trips",'orientation':"vertical"})

ax.set_title("NYC Pickup Trips by Zone", fontsize=15)
plt.show()
```

Map:



Outcome: Displays the number of trips by zone

3.1.14. Conclude with results

- Taxi demand is high during peak hours, i.e., 9:00 AM to 7:00 PM.
- Q2 and Q4 had higher revenue compared to other quarters.
- As the trip distance increases, the fare amount also increases.
- The preferred payment mode is Credit Card.
- Tip amount increases with trip distance.
- Fare amount and passenger count are not correlated.
- May and October had the highest revenue compared to other months.
- LocationID 263 is the busiest zone; increasing the number of taxis in this zone can accommodate the surge in demand.
- Fare amount and trip duration are not highly correlated, which suggests there could be improvements by factoring in wait time or congestion time when calculating fares.

3.2. Detailed EDA: Insights and Strategies

3.2.1. Identify slow routes by comparing average speeds on different routes

Code:

```
new_df["route"]=new_df["PULocationID"].astype('str')+"->"+new_df["DOLocationID"].astype(str)
new_df["trip_duration"]=(new_df["tpep_dropoff_datetime"]-new_df["tpep_pickup_datetime"]).dt.total_seconds()/3600
tripDetails=new_df.groupby(["route","trip_distance","pickup_hour"])["trip_duration"].mean().reset_index()

tripDetails["speed"]=tripDetails["trip_distance"]/tripDetails["trip_duration"]
tripDetails=tripDetails[tripDetails["trip_duration"]>0]
tripDetails.sort_values("speed")
tripDetails.loc[tripDetails.groupby('route')['speed'].idxmin()]
```

Result: Slowest route is 1->1 where speed is 0.30mile/hour

	route	trip_distance	pickup_hour	trip_duration	speed
1	1->1	0.01	11	0.032500	0.307692
3	1->265	0.07	16	0.003889	18.000000
4	10->100	15.47	20	0.877778	17.624051
5	10->13	19.69	16	0.908056	21.683695
6	10->131	6.25	15	0.359167	17.401392
...	...	...	...	...	...
211816	97->65	0.56	21	0.055833	10.029851
211818	97->66	1.30	19	0.173889	7.476038
211820	97->68	4.99	21	0.404722	12.329444
211821	97->76	6.12	20	0.494444	12.377528

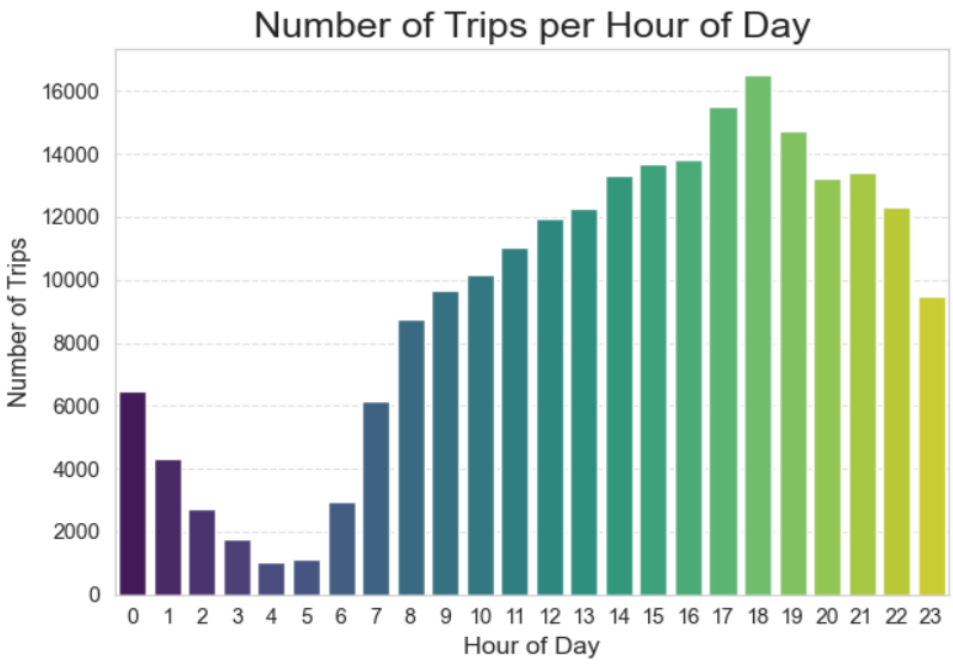
3.2.2. Calculate the hourly number of trips and identify the busy hours

Visualising the number of trips per hour

Code:

```
taxiPickups=new_df["pickup_hour"].value_counts().sort_index()
sns.barplot(x=taxiPickups.index, y=taxiPickups.values, palette='viridis')
plt.title('Number of Trips per Hour of Day', fontsize=18)
plt.xlabel('Hour of Day', fontsize=12)
plt.ylabel('Number of Trips', fontsize=12)
plt.xticks(range(24))
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```

Graph:



Calculating the busiest hour and the number of trip

Busiest hour: 18:00 with 16497 trips

```
#busiest Hour
busiest_hour = taxiPickups.idxmax()
trip_count = taxiPickups.max()
print(f"Busiest hour: {busiest_hour}:00 with {trip_count} trips")
```

```
Busiest hour: 18:00 with 16497 trips
```

3.2.3. Scale up the number of trips from above to find the actual number of trips

Sample fraction was 0.78%

ScaleFactor=1/Sample Fraction

Top 5 busiest hours with number of trips

18:00 2.115000e+06

17:00 1.990769e+06

19:00 1.889872e+06

16:00 1.769615e+06

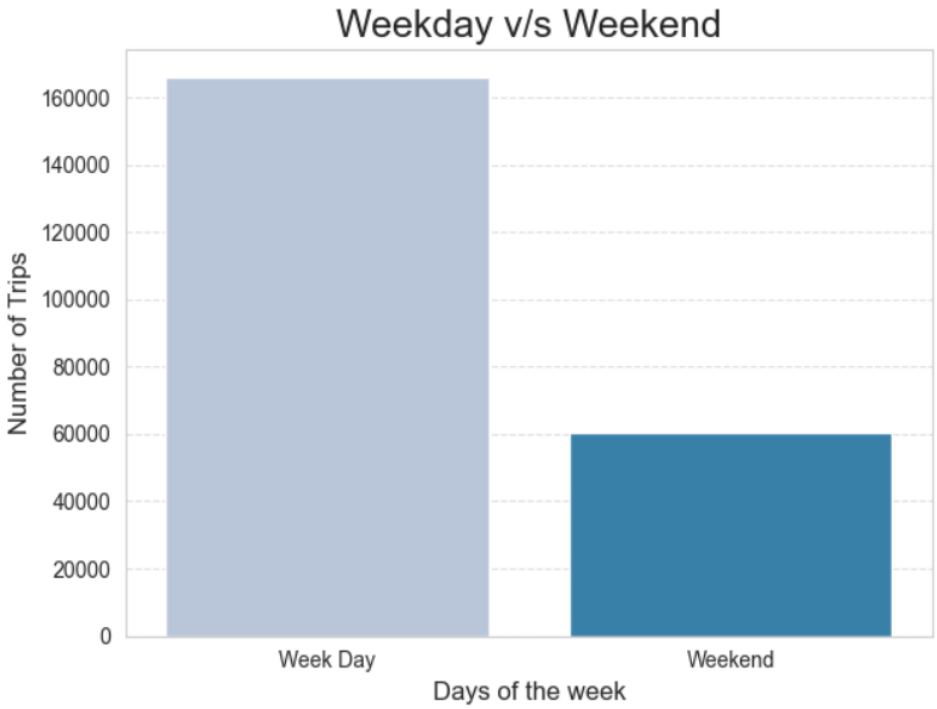
15:00 1.753718e+06

3.2.4. Compare hourly traffic on weekdays and weekends

Code:

```
new_df["week_day_weekend"]=new_df["pickup_day_of_the_week"].map(lambda x:"Week Day" if x in [0,1,2,3,4] else "Weekend" )
weekdayComparison=new_df["week_day_weekend"].value_counts()
sns.barplot(x=weekdayComparison.index, y=weekdayComparison.values, palette='PuBu')
plt.title('Weekday v/s Weekend', fontsize=18)
plt.xlabel('Days of the week', fontsize=12)
plt.ylabel('Number of Trips', fontsize=12)
plt.xticks(range(2))
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```

Graph:



Result: Number of trips on week day is more than the Weekend

3.2.5. Identify the top 10 zones with high hourly pickups and drops

Code:

```
drop_location_df=pd.merge(new_df, zones, left_on='DOLocationID', right_on='LocationID', how='inner')
dropZones=drop_location_df['LocationID'].value_counts().sort_index()
zones["dropped_trips"]=zones["LocationID"].map(dropZones)
print("top 10 pickup zones\n", zones.sort_values("trips",ascending=False)[["zone","trips"]].head(10))
print("\n")
print("top 10 drop off zones\n", zones.sort_values("dropped_trips",ascending=False)[["zone","dropped_trips"]].head(10))
```

Result:

Top 10 pickup zones

```
top 10 pickup zones
                              zone    trips
236         Upper East Side South  11148.0
160                Midtown Center  10640.0
235         Upper East Side North  10150.0
131                   JFK Airport  10038.0
161                  Midtown East   8450.0
137             LaGuardia Airport   8004.0
141           Lincoln Square East   7723.0
185  Penn Station/Madison Sq West   7587.0
229     Times Sq/Theatre District   7025.0
169                   Murray Hill   6828.0
```

Top 10 drop off zones

```
top 10 drop off zones
                           zone  dropped_trips
235      Upper East Side North        10809.0
236      Upper East Side South         9855.0
160             Midtown Center         8907.0
169                Murray Hill         6901.0
238      Upper West Side South         6805.0
141        Lincoln Square East         6644.0
161                Midtown East         6586.0
140            Lenox Hill West         6321.0
229  Times Sq/Theatre District         6162.0
67                 East Chelsea         5823.0
```

3.2.6. Find the ratio of pickups and dropoffs in each zone

Code:

```
zones["ratio_pickup_drop"]=zones["trips"]/zones["dropped_trips"]
print("top 10 highest ratio zones\n", zones.sort_values("ratio_pickup_drop",ascending=False)[["zone","ratio_pickup_drop"]].head(10))
print("\n")
print("top 10 lowest ratio zones\n", zones.sort_values("ratio_pickup_drop",ascending=True)[["zone","ratio_pickup_drop"]].head(10))
```

Result:

Top 10 pickups/drop ratio in zone

```
top 10 highest ratio zones
                            zone  ratio_pickup_drop
69                 East Elmhurst          13.421053
131                  JFK Airport           4.317419
137            LaGuardia Airport           2.591969
185  Penn Station/Madison Sq West           1.575374
214                South Jamaica           1.533333
42                  Central Park           1.381499
248                 West Village           1.374594
113       Greenwich Village South           1.344828
161                 Midtown East           1.283025
99              Garment District           1.215835
```

Lowest 10 pickups/drop ratio zone

```
lowest 10 ratio zones
                    zone  ratio_pickup_drop
0         Newark Airport           0.006116
13             Bay Ridge           0.008197
256      Windsor Terrace           0.009615
91              Flushing           0.011905
201    Roosevelt Island           0.016529
195            Rego Park           0.024390
224  Stuyvesant Heights           0.028736
197            Ridgewood           0.029126
204         Saint Albans           0.031250
122            Homecrest           0.033333
```

3.2.7. Identify the top zones with high traffic during night hours

Code:

```
pickupInNight=merge_df[(merge_df['pickup_hour']<=5)| (merge_df['pickup_hour']==23)]
pickupInNight['zone'].value_counts()
print("Top 10 zone pick up during night hours \n",pickupInNight['zone'].value_counts().head(10))
print("\n")
drop_location_df['drop_hour']=drop_location_df["tpep_dropoff_datetime"].dt.hour
dropInNight=drop_location_df[(drop_location_df['drop_hour']<=5)| (drop_location_df['drop_hour']==23)]
print("Top 10 drop off zone during night hours \n",dropInNight['zone'].value_counts().head(10))
```

Result:

Top 10 zone pick up during night hours

```
Top 10 zone pick up during night hours
 zone
East Village                    2079
West Village                    1729
JFK Airport                     1524
Lower East Side                 1312
Clinton East                    1260
Greenwich Village South         1121
Times Sq/Theatre District        983
Penn Station/Madison Sq West     843
LaGuardia Airport                769
Gramercy                         750
Name: count, dtype: int64
```

Top 10 drop off zone during night hours

```
Top 10 drop off zone during night hours
 zone
East Village             1167
Clinton East              901
Murray Hill               862
Gramercy                  812
Lenox Hill West           807
East Chelsea              794
Yorkville West            771
Upper East Side North     717
West Village              698
Upper West Side South     667
```

3.2.8. Find the revenue share for nighttime and daytime hours

Code:

```
# Filter for night hours (11 PM to 5 AM)
pickupInNight['total_amount'].sum()
dayTimeRides=merge_df[~((merge_df['pickup_hour']<=5)| (merge_df['pickup_hour']==23))]
dayTimeRides["total_amount"].sum().round(2)

print("revenue share for night time ",pickupInNight['total_amount'].sum().round(2))
print("\n")
print("revenue share for day time ",dayTimeRides['total_amount'].sum().round(2))
```

Result:

revenue share for night time is 805383.57

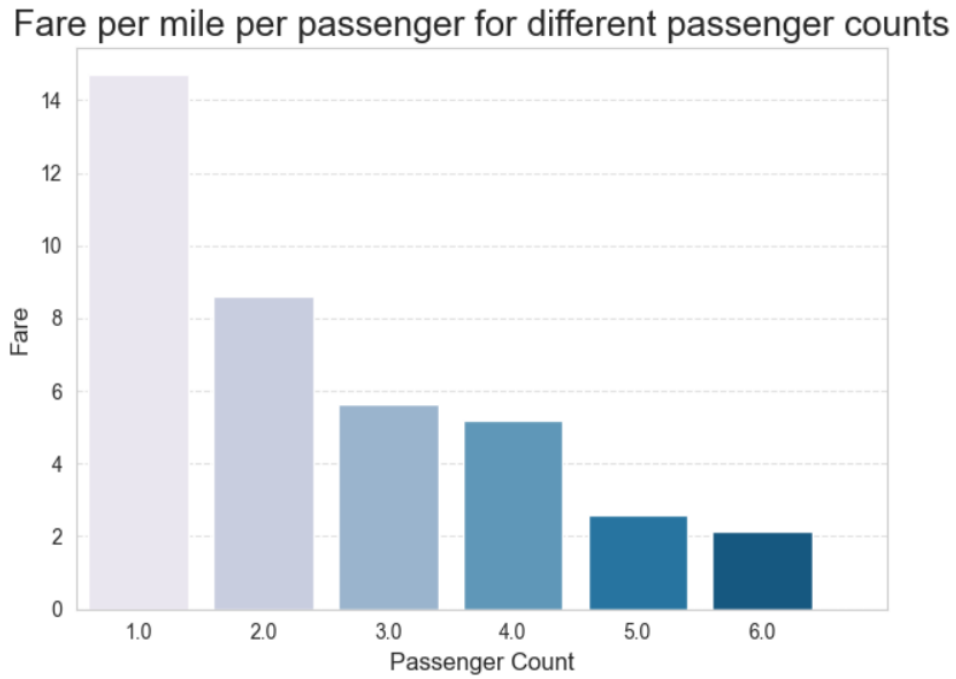
revenue share for day time is 5834403.93

3.2.9. For the different passenger counts, find the average fare per mile per passenger

Code:

```
# Analyse the fare per mile per passenger for different passenger counts
merge_df["average_fare_per_mile"]=merge_df['total_amount']/merge_df['trip_distance']
merge_df["average_fare_per_mile_pass"]=merge_df["average_fare_per_mile"]/merge_df['passenger_count']
merge_df[['total_amount','passenger_count','average_fare_per_mile','average_fare_per_mile_pass']]
avgFarePersonPerMile=merge_df.groupby("passenger_count")["average_fare_per_mile_pass"].mean()
sns.barplot(x=avgFarePersonPerMile.index, y=avgFarePersonPerMile.values, palette='PuBu')
plt.title('Fare per mile per passenger for different passenger counts', fontsize=18)
plt.xlabel('Passenger Count', fontsize=12)
plt.ylabel('Fare', fontsize=12)
plt.xticks(range(7))
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```

Visualisation:



Result :

```
passenger_count
1.0    14.689415
2.0     8.587225
3.0     5.627111
4.0     5.209289
5.0     2.575920
6.0     2.146256
```

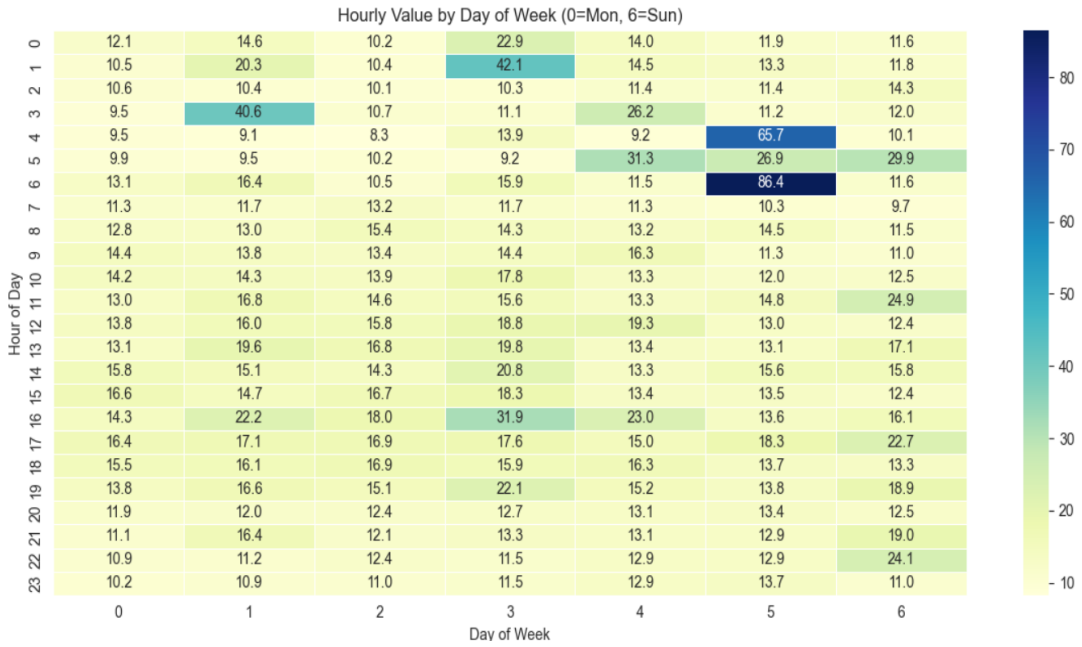
These are the average fare per mile for passenger count

3.2.10. Find the average fare per mile by hours of the day and by days of the week

Code:

```
comparisonWithDays=merge_df.groupby(["pickup_day_of_the_week","pickup_hour"])["average_fare_per_mile"].mean().reset_index()
pivotDf= comparisonWithDays.pivot_table(
    index='pickup_hour',
    columns='pickup_day_of_the_week',
    values='average_fare_per_mile'
)
plt.figure(figsize=(12, 6))
sns.heatmap(pivotDf,cmap='YlGnBu',annot=True, fmt=".1f",linewidths=0.5)
plt.title('Hourly Value by Day of Week')
plt.xlabel('Day of Week')
plt.ylabel('Hour of Day')
plt.tight_layout()
plt.show()
```

Result:

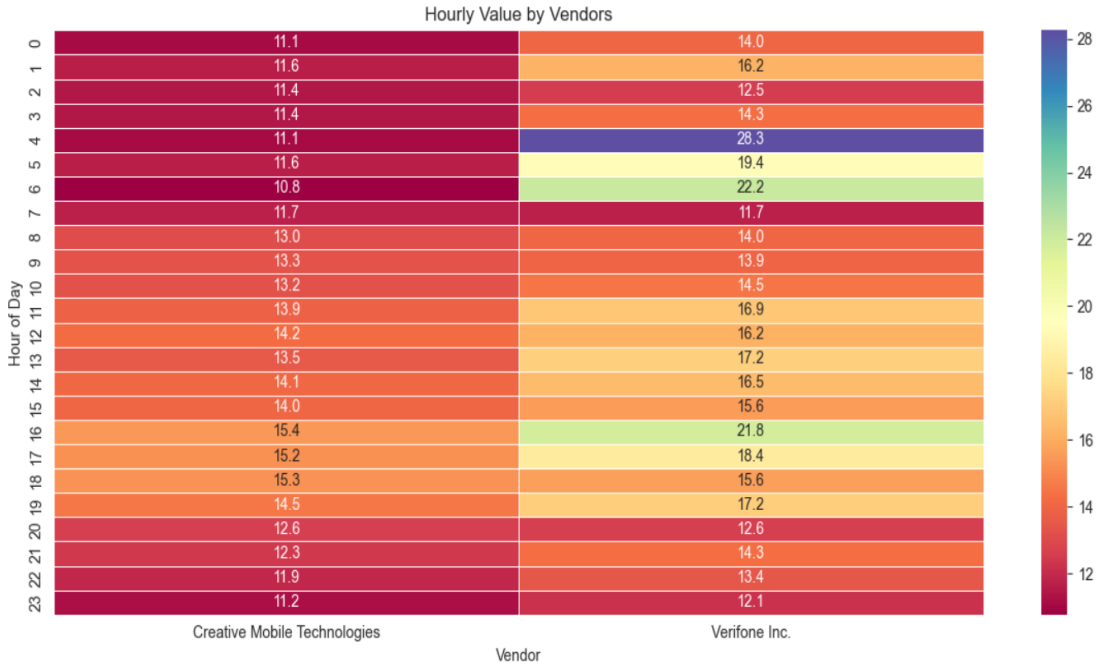


3.2.11. Analyse the average fare per mile for the different vendors

Code:

```
comparisonWithVendors=merge_df.groupby(["VendorID","pickup_hour"])["average_fare_per_mile"].mean().reset_index()
vendorNames = ['Creative Mobile Technologies','Verifone Inc.']
pivotDf= comparisonWithVendors.pivot_table(
    index='pickup_hour',
    columns='VendorID',
    values='average_fare_per_mile'
)
plt.figure(figsize=(12, 6))
sns.heatmap(pivotDf,cmap='Spectral',annot=True, fmt=".1f",linewidths=0.5,xticklabels=vendorNames)
plt.title('Hourly Value by Vendors')
plt.xlabel('Vendor')
plt.ylabel('Hour of Day')
plt.tight_layout()
plt.show()
```

Result through HeatMap



3.2.12. Compare the fare rates of different vendors in a distance-tiered fashion

Code:

```
vendor_map = {
    1: 'Creative Mobile Technologies',
    2: 'VeriFone Inc.'
}
def get_tiers(val):
    if val <= 2:
        return '0-2 miles'
    elif val <= 5:
        return '2-5 miles'
    else:
        return '5+ miles'
merge_df["distance_tiers"]=merge_df["trip_distance"].apply(get_tiers)
distanceTierDf=merge_df.groupby(["VendorID","distance_tiers"])["average_fare_per_mile"].mean().reset_index()
distanceTierDf["vendorName"]=distanceTierDf["VendorID"].map(vendor_map)
pivotView=distanceTierDf.pivot_table( index='distance_tiers',
    columns='vendorName',
    values='average_fare_per_mile')
print(pivotView)
```

Result: (Pivot Table)

vendorName	Creative Mobile Technologies	VeriFone Inc.
distance_tiers		
0–2 miles	17.144266	22.106027
2–5 miles	9.518241	9.716557
5+ miles	6.381656	6.406772

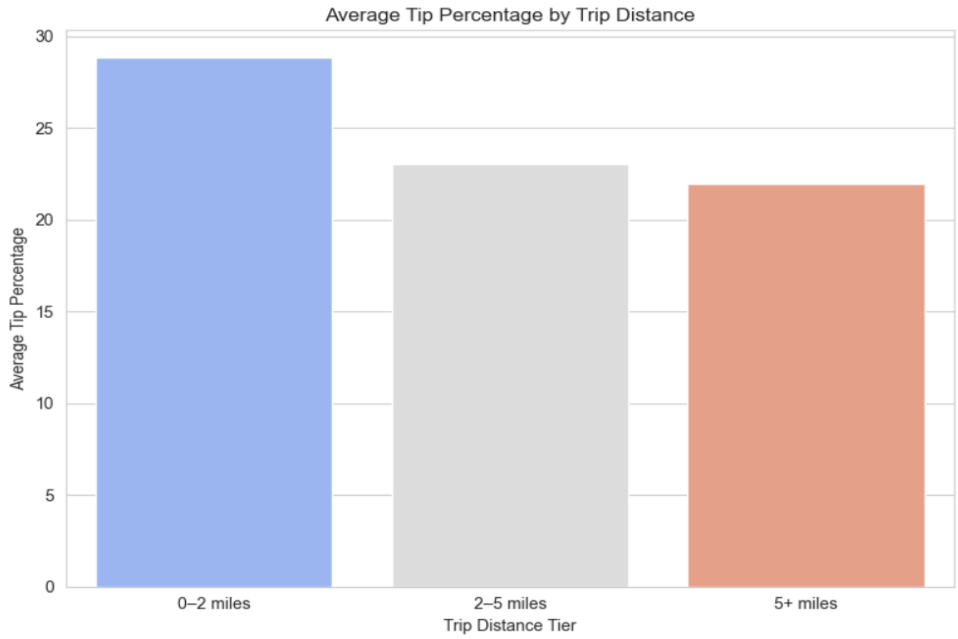
3.2.13. Analyse the tip percentages

Average tip by distance tier

Code:

```
merge_df["tip_percent"]=(merge_df['tip_amount'] / merge_df['fare_amount']) * 100

# tip percentage by Trip distance
distanceVsTip=merge_df.groupby('distance_tiers')['tip_percent'].mean().reset_index()
plt.figure(figsize=(8, 6))
sns.barplot(data=distanceVsTip, x='distance_tiers', y='tip_percent', palette='coolwarm')
plt.title('Average Tip Percentage by Trip Distance')
plt.xlabel('Trip Distance Tier')
plt.ylabel('Average Tip Percentage')
plt.tight_layout()
plt.show()
```

Result:

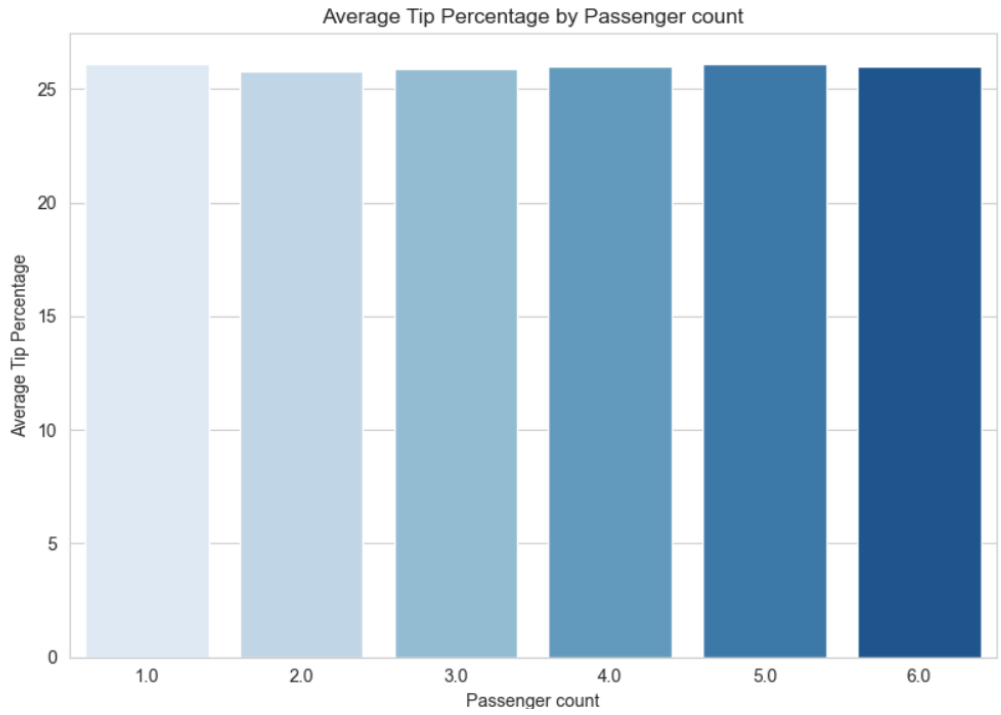


Average tip by Passenger count

Code:

```
distanceVsTip=merge_df.groupby('passenger_count')['tip_percent'].mean().reset_index()
plt.figure(figsize=(8, 6))
sns.barplot(data=distanceVsTip, x='passenger_count', y='tip_percent', palette='Blues')
plt.title('Average Tip Percentage by Passenger count')
plt.xlabel('Passenger count')
plt.ylabel('Average Tip Percentage')
plt.tight_layout()
plt.show()
```

Result:



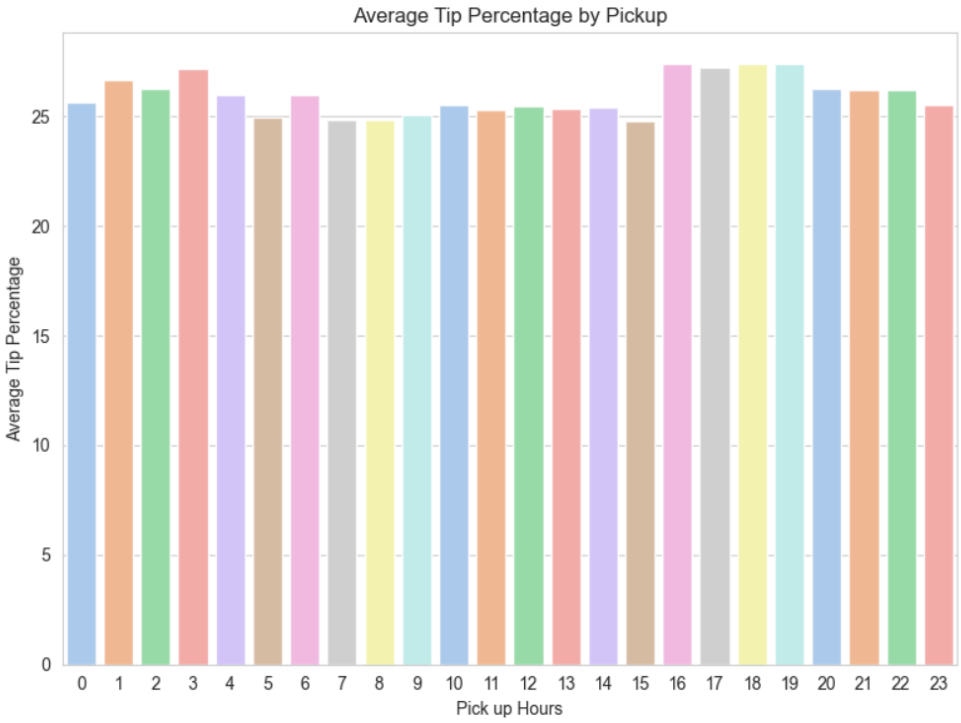
Outcome: Display tip is not affected by passenger count

Average tip by pickup hour

Code:

```
distanceVsTip=merge_df.groupby('pickup_hour')['tip_percent'].mean().reset_index()
plt.figure(figsize=(8, 6))
sns.barplot(data=distanceVsTip, x='pickup_hour', y='tip_percent', palette='pastel')
plt.title('Average Tip Percentage by Pickup')
plt.xlabel('Pick up Hours')
plt.ylabel('Average Tip Percentage')
plt.tight_layout()
plt.show()
```

Result:



Outcome: Average tip doesn't have much impact by pickup hours

Comparing the tip percentage

```
# Compare trips with tip percentage < 10% to trips with tip percentage > 25%
tripsWithlowPercentage=len(merge_df[merge_df['tip_amount']<10])
tripsWithHighPercentage=len(merge_df[merge_df['tip_amount']>25])
tippingData = {
    'tip percentage < 10%': [tripsWithlowPercentage],
    'tip percentage > 25%': [tripsWithHighPercentage]
}
tip_df=pd.DataFrame(tippingData)
print(tip_df)
```

```
   tip percentage < 10%  tip percentage > 25%
0                203526                   493
```

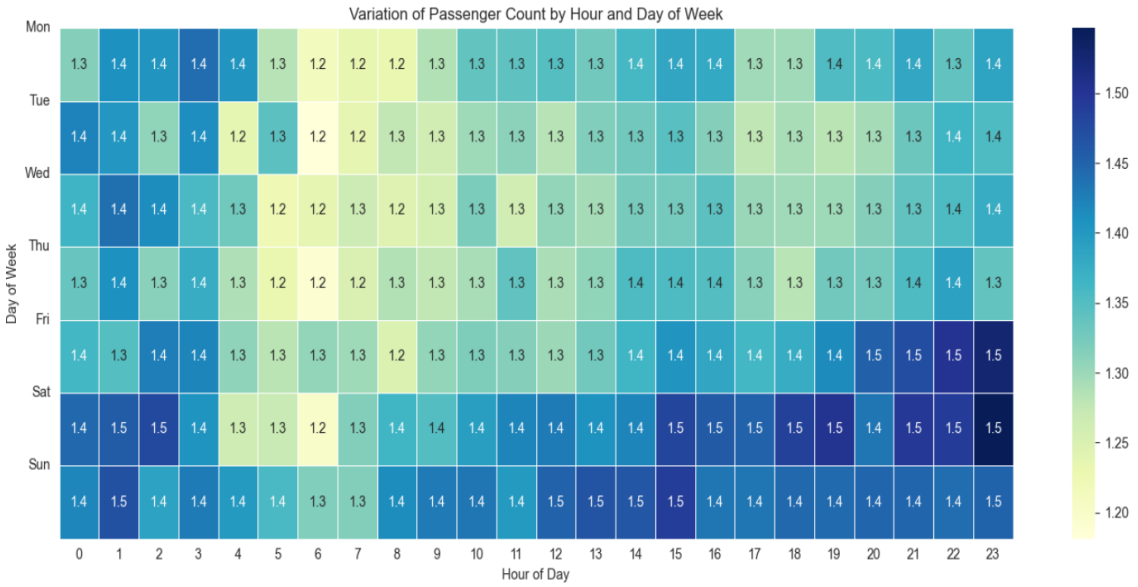
3.2.14. Analyse the trends in passenger count

Graph to display how Passenger count varies across hours and days

Code:

```
grouped_df = merge_df.groupby(['pickup_day_of_the_week', 'pickup_hour'])['passenger_count'].mean().reset_index()
pivot_table = grouped_df.pivot(index='pickup_day_of_the_week', columns='pickup_hour', values='passenger_count')
plt.figure(figsize=(14, 6))
sns.heatmap(pivot_table, cmap='YlGnBu', linewidths=0.5, annot=True, fmt=".1f")
plt.title('Variation of Passenger Count by Hour and Day of Week')
plt.xlabel('Hour of Day')
plt.ylabel('Day of Week')
plt.yticks(ticks=range(7), labels=['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun'], rotation=0)
plt.tight_layout()
plt.show()
```

Result:



Outcome: It shows the average passenger count is more on weekends

3.2.15. Analyse the variation of passenger counts across zones

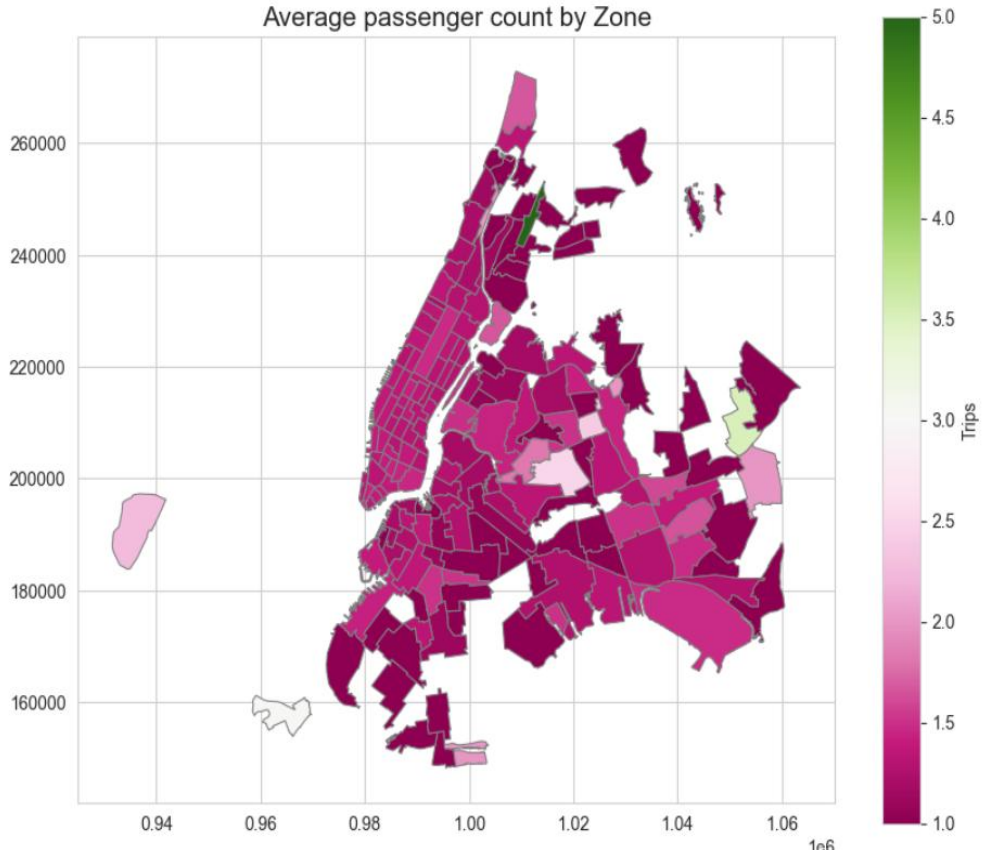
Code:

```
zones_df = merge_df.groupby(['zone'])['passenger_count'].mean().reset_index()
print("top 10 zones with max avg passenger_count \n",zones_df.sort_values('passenger_count',ascending=False).head(10))
```

Result: Showing the top 10 zones with max avg Passenger count

```
top 10 zones with max avg passenger_count
                          zone  passenger_count
26          Claremont/Bathgate             5.00
114            Oakland Gardens             3.50
1      Arrochar/Fort Wadsworth             3.00
102             Middle Village             2.50
34                      Corona             2.40
112             Newark Airport             2.25
96             Manhattan Beach             2.00
72             Highbridge Park             2.00
168              Willets Point             2.00
125             Queens Village             2.00
```

Graph:



3.2.16. Analyse the pickup/dropoff zones or times when extra charges are applied more frequently.

Average surcharge based on the pickup zones

```
merge_df['surcharge_total']=merge_df['improvement_surcharge']+merge_df['congestion_surcharge']+merge_df['extra']
print("Average surcharge based on pickup zones\n", merge_df.groupby('zone')['surcharge_total'].mean().reset_index().sort_values('surcharge_total',ascer
```

Result

```
Average surcharge based on pickup zones
                          zone  surcharge_total
87           LaGuardia Airport         8.997164
43               East Elmhurst         7.526765
25                 City Island         6.000000
103              Midtown Center         5.171194
153  Times Sq/Theatre District         5.162961
6             Battery Park City         5.087861
104                Midtown East         5.087799
105               Midtown North         5.070215
156         UN/Turtle Bay South         5.052090
157                    Union Sq         5.036641
```

Average surcharge based on drop zones

Code:

```
drop_location_df['surcharge_total']=drop_location_df['improvement_surcharge']+drop_location_df['congestion_surcharge']+drop_location_df['extra']
print("Average surcharge based on Drop zones\n", drop_location_df.groupby('zone')['surcharge_total'].mean().reset_index().sort_values('surcharge_total',a
```

Result:

```
Average surcharge based on Drop zones
                              zone  surcharge_total
124               LaGuardia Airport         8.518556
168                     Pelham Bay         7.000000
71                    East Tremont         6.916667
5                     Astoria Park         6.000000
235                  Willets Point         6.000000
139                    Marble Hill         6.000000
169                 Pelham Bay Park         6.000000
114                Inwood Hill Park         5.708333
104     Heartland Village/Todt Hill         5.500000
206  Stuy Town/Peter Cooper Village         5.346922
```

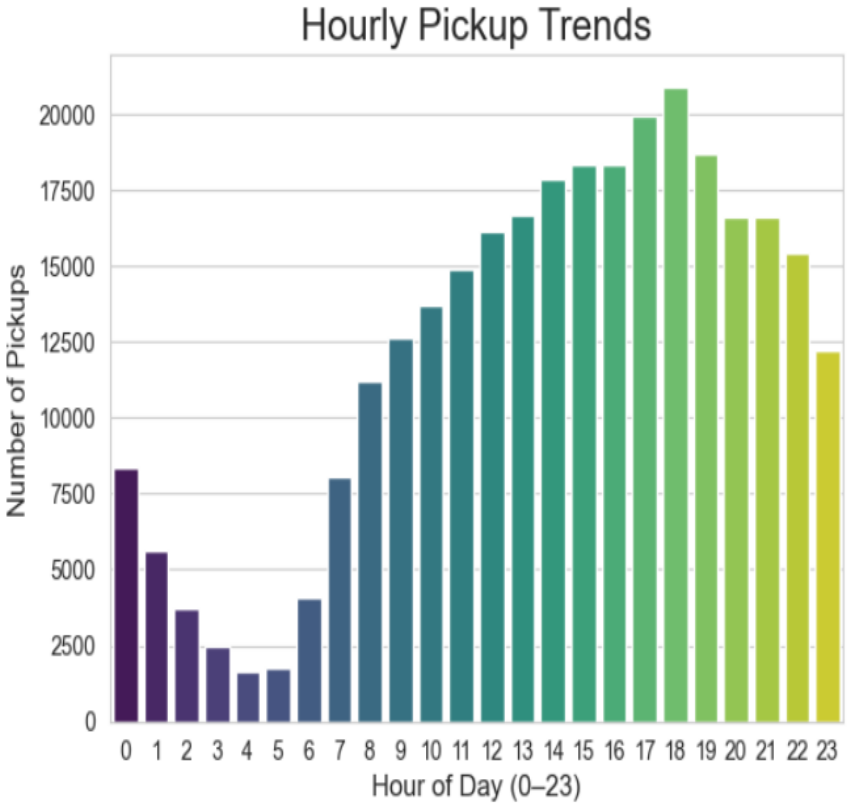
4. Conclusions

4.1. Final Insights and Recommendations

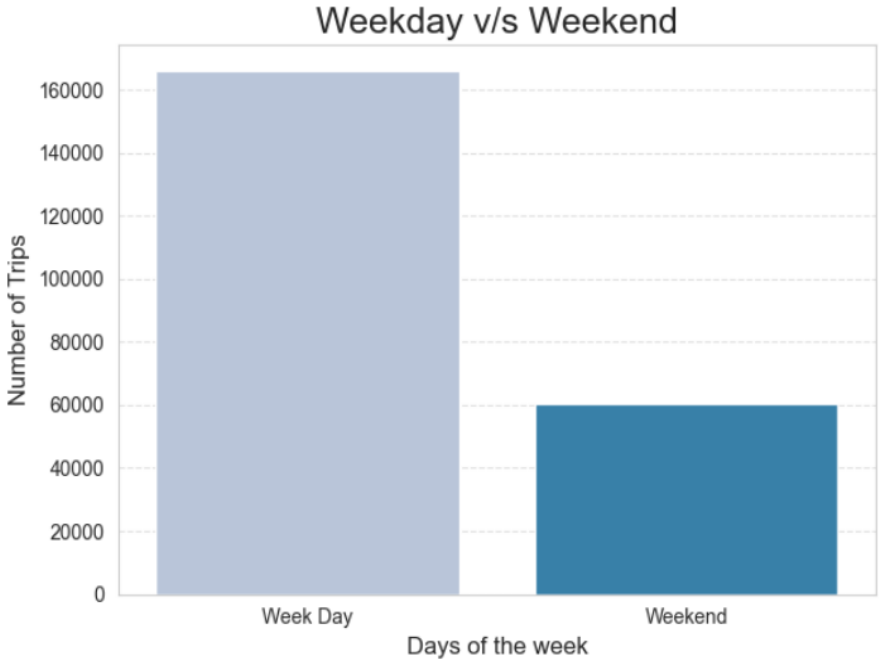
4.1.1. Recommendations to optimize routing and dispatching based on demand patterns and operational inefficiencies.

Time of the day Startergy: As hourly pickup trend shows the demand is more in the peak hours that is 9:00AM to 6:00PM in the evening. We

need to increase the number of cabs to accomodate the surge in demand.
The below Chart visualizes it more clearly



Days of the week strategy: As we see the number of trips during weekdays are more as compared to weekends. Then We should increase focus on deploying more taxi of weedays



Deploying SUV's in zone where the demand of passenger count is more Like deploying it in these zones

top 10 zones with max avg passenger_count

	zone	passenger_count
26	Claremont/Bathgate	5.00
114	Oakland Gardens	3.50
1	Arrochar/Fort Wadsworth	3.00
102	Middle Village	2.50
34	Corona	2.40
112	Newark Airport	2.25
96	Manhattan Beach	2.00
72	Highbridge Park	2.00
168	Willets Point	2.00
125	Queens Village	2.00

We can charge a higher fare amount for SUV taxis to increase revenue

4.1.2. Suggestions on strategically positioning cabs across different zones to make best use of insights uncovered by analysing trip trends across time, days and months.

Increasing the number of taxis in these zones will help accommodate the demand.

top 10 pickup zones

	zone	trips
236	Upper East Side South	11148.0
160	Midtown Center	10640.0
235	Upper East Side North	10150.0
131	JFK Airport	10038.0
161	Midtown East	8450.0
137	LaGuardia Airport	8004.0
141	Lincoln Square East	7723.0
185	Penn Station/Madison Sq West	7587.0
229	Times Sq/Theatre District	7025.0
169	Murray Hill	6828.0

top 10 drop off zones

	zone	dropped_trips
235	Upper East Side North	10809.0
236	Upper East Side South	9855.0
160	Midtown Center	8907.0
169	Murray Hill	6901.0
238	Upper West Side South	6805.0
141	Lincoln Square East	6644.0
161	Midtown East	6586.0
140	Lenox Hill West	6321.0
229	Times Sq/Theatre District	6162.0
67	East Chelsea	5823.0

By Time of the Day

Top zones with demand based on hour

	zone	pickup_hour	trip_count
1345	Midtown Center	18	973
1344	Midtown Center	17	960
1959	Upper East Side South	17	850
1933	Upper East Side North	15	843
1957	Upper East Side South	15	837
1960	Upper East Side South	18	831
1346	Midtown Center	19	827
1956	Upper East Side South	14	825
1958	Upper East Side South	16	815
1935	Upper East Side North	17	796

Based on the above inference, increasing the number of taxis in these zones will help accommodate the surge and generate more revenue.

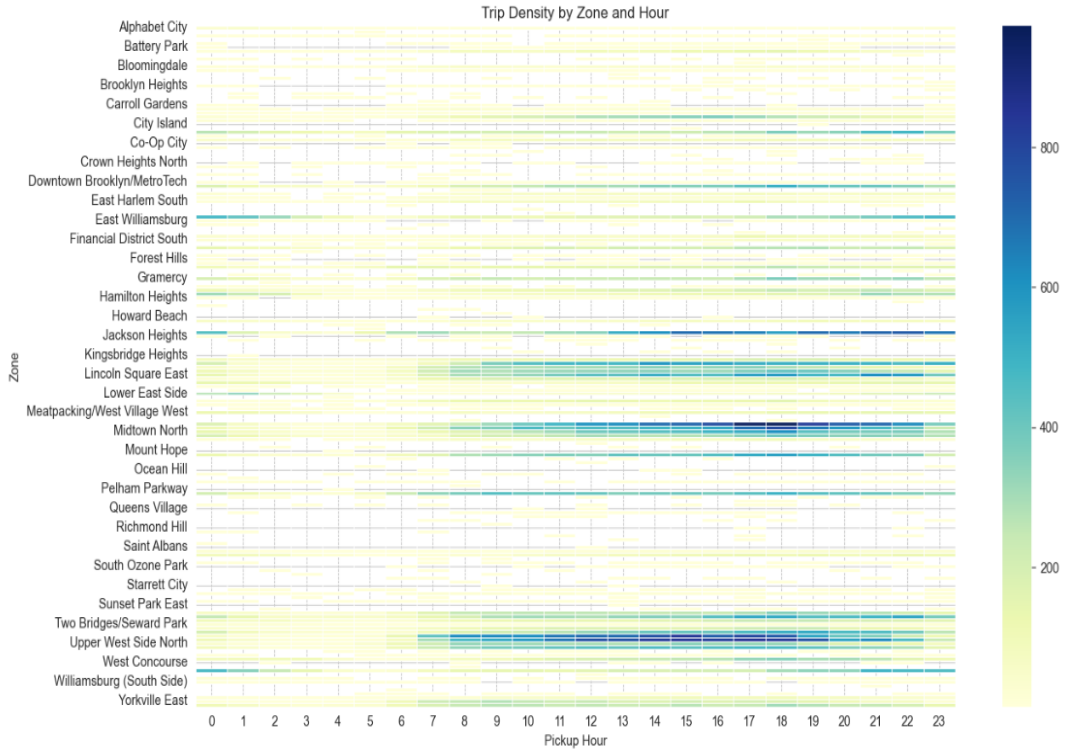
By Month

Top zones with demand based on month

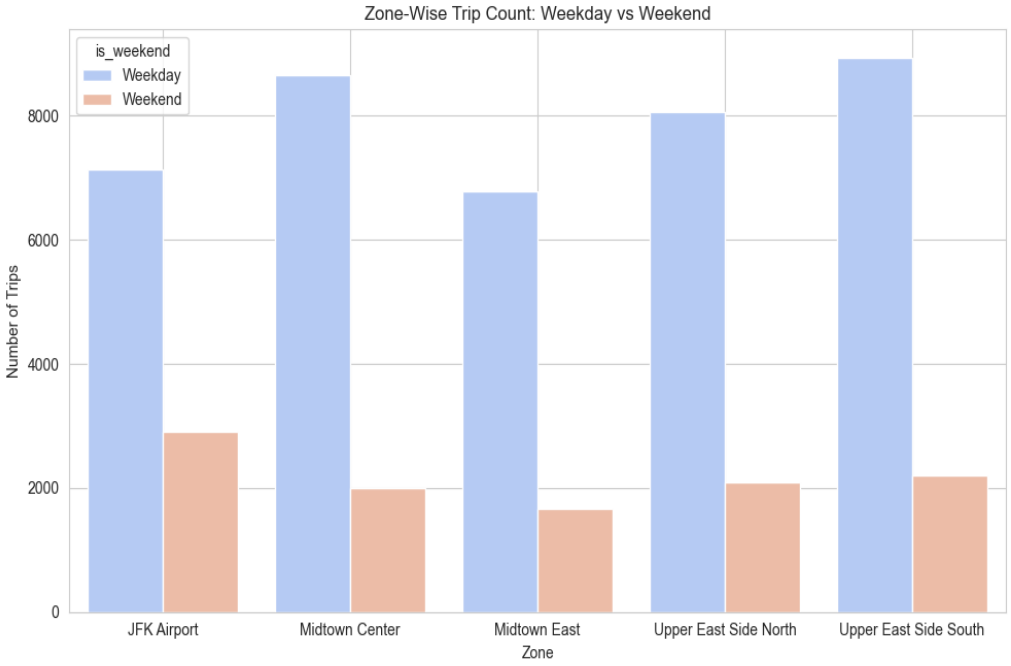
	zone	monthly_pickups	trip_count
1152	Upper East Side South	5	1076
788	Midtown Center	11	1052
1158	Upper East Side South	11	1051
1146	Upper East Side North	11	1049
1157	Upper East Side South	10	1033
1150	Upper East Side South	3	1019
1159	Upper East Side South	12	1014
787	Midtown Center	10	1004
1151	Upper East Side South	4	990
1140	Upper East Side North	5	980

We can deploy more taxi in the winter months to these zones "Upper East Side South","Upper East Side North","Midtown Center"

Busiest Zones



Zones demand weekday/weekend

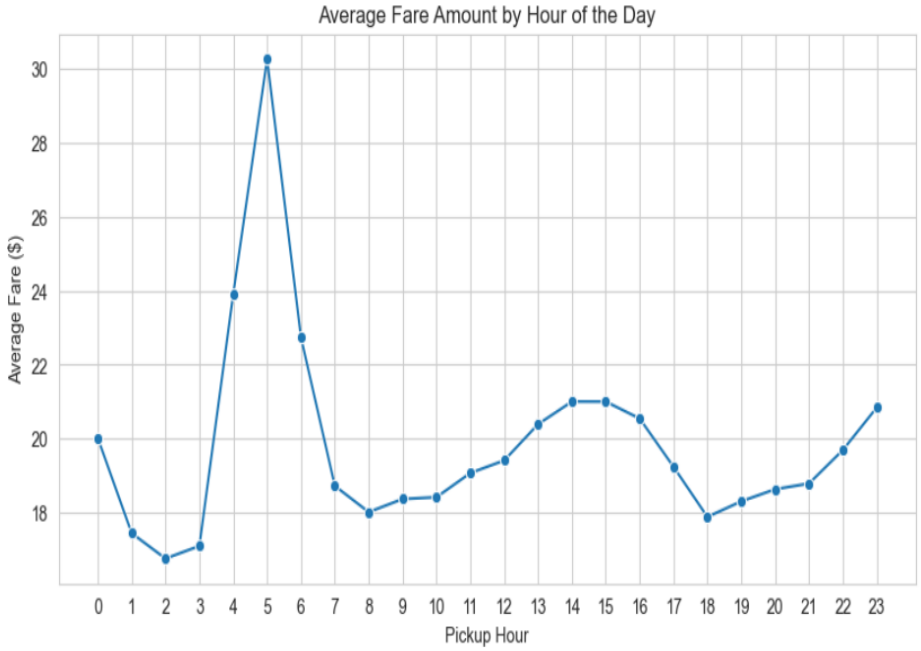


4.1.3. Propose data-driven adjustments to the pricing strategy to maximize revenue while maintaining competitive rates with other vendors.

Dynamic Pricing:

- Implement Dynamic pricing based on real time supply and demand. We need to increase the price is where the demand of taxi's is high

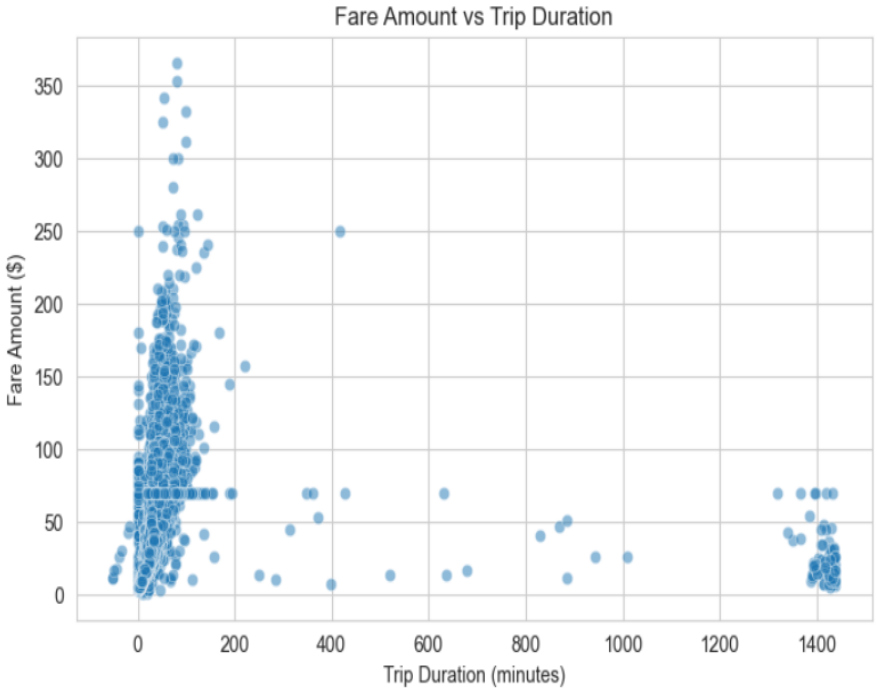
  Average Fare by Hour (for Dynamic Pricing)



Time of Day Pricing:

- We can addjust prices based on the time of day. Increase fares during rush hours 9:00AM to 6:00PM when demand is high. Offer discounts during night hours 11:00PM to 5:00AM.

  # Fare amount vs trip duration



Tier Pricing

- Based on the distance of the trip we can implement tiered pricing. For shorter distance ride we can charge a premium and over 5 miles we can over slight discount.
  Distance tiers